Information Systems Audit Assignment - Alice Malambo (2406344266)

Name	Alice Malambo
SIN	2406344266
Course	Information Systems Audit
Programme	IT in Business Management
Email	malamboalice78@gmail.com
Institution	Information and Communications University – School of Engineering
Due date	10 November 2026

Workstation: Linux forensic workstation, case folder and Autopsy case `AliceMalambo_2406344266`. Tools: Autopsy, The Sleuth Kit (fsstat, fls, istat, icat, ifind, mmls), libewf (ewfinfo, ewfverify), dcfldd, xxd, sqlite3, RegRipper / python-registry, 7-Zip, ExifTool, OpenStego.

Executive summary

- **Image integrity:** the laptop image (MD5 `e6966b68…f08a`, SHA1 `22a24339…81b3`) and the USB image (MD5 `63e51c59…f959`) both match the acquisition hashes, so neither was altered.
- **The laptop:** a Windows 10 Pro virtual machine. Its single user, **Charles Kingston**, is a **local administrator**.
- **29 March 2022 activity:** he searched Google Maps for hotels near Knossos. He focused on **Galaxy Hotel Iraklio**, and planned a route from there to Knossos. He read about an F-35 military deal. Then he searched for, downloaded and installed the steganography tool **OpenStego 0.8.4**.
- **No VPN, Tor, encrypted messengers, remote-access tools or VeraCrypt/BitLocker volumes** were found.
- **USB drive:** a “deleted” `pass.txt` was recovered by editing the FAT32 directory entry. It holds his Facebook, e-mail and **archive** passwords.
- **The archive:** the archive password decrypts the AES-256 **vacation.zip**. It contains:
  - a **DJI drone photo** taken on **19 Feb 2022** at **35.330057 N, 24.998574 E**, in the hills west of Heraklion, showing a remote new road;
  - **knossos.png**, a probable OpenStego carrier.
- **Conclusion:** the evidence contradicts the tourist story and is consistent with espionage tradecraft.

> **Screenshots:** terminal screenshots of every CLI step are included at the end of each part (Figures 1–22). Autopsy GUI and OpenStego GUI screenshots must be added from a desktop session.

Part 1 – Computer Analysis (laptop.E01)

> Student: Alice Malambo (SIN 2406344266). Create the Autopsy case as **`AliceMalambo_2406344266`** inside the folder `AliceMalambo_2406344266`, add `laptop.E01` as a *Disk Image* data source, and run the default ingest modules (Recent Activity, Hash Lookup, File Type ID, Extension Mismatch, Embedded File Extractor, EXIF Parser, Keyword Search, Encryption Detection). Take a screenshot of each Autopsy view referenced below.

1. Verify the integrity of the forensic image

a/b. Commands (Linux, or FTK Imager → *Verify Drive/Image* on Windows):

```
        ewfinfo   laptop.E01
        ewfverify -d sha1 laptop.E01
```

```
MD5 hash stored in file:          e6966b688d3c3d7be5d58a90dd6df08a
MD5 hash calculated over data:    e6966b688d3c3d7be5d58a90dd6df08a
SHA1 hash stored in file:         22a2433980782220b38caca3cf2279f0f2a981b3
SHA1 hash calculated over data:   22a2433980782220b38caca3cf2279f0f2a981b3
ewfverify: SUCCESS
```

	Calculated	acquisition_details
MD5	e6966b688d3c3d7be5d58a90dd6df08a	e6966b688d3c3d7be5d58a90dd
SHA1	22a2433980782220b38caca3cf2279f0f2a981b3	22a2433980782220b38caca3cf2 (labelled “SHA256” in the file, bu digit value is a SHA-1)

The hashes are computed over the 50 GiB of acquired data, not over the `.E01`–`.E08` container files. They match the values recorded by FTK Imager 4.7.1.2 at acquisition time (29 Mar 2022, examiner D. Kalochristianakis). The image was therefore **not altered** after acquisition. Autopsy shows the same check under *Data Source → right-click → Run Ingest → Data Source Integrity*.

c. Why hash verification matters in an IT audit: - **Integrity and admissibility:** it proves that the evidence analysed is bit-for-bit identical to what was seized. Without this, findings can be challenged as tampered with or contaminated. - **Chain of custody:** each hand-over can be checked against the original hash, so accountability is preserved. - **Repeatability:** a second auditor can reproduce the results from the same verified data. This supports independence and objectivity (ISACA ITAF / IS Audit Standard 1205 *Evidence*). - **Detects silent corruption** from transfers, storage or faulty media. - **Regulatory requirements:** ISO/IEC 27037 and ISO 27001 A.5.28 (collection of evidence) require evidence to be collected and kept in a verifiable state.

2. Windows user account

Autopsy: *Data Artifacts → Operating System User Account* and *OS Info*. CLI: `fls`/`icat` to pull `Windows/System32/config/SAM`, then open it with RegRipper (`rip.pl -r SAM -p samparse`) or Registry Explorer.

Item	Value
OS	Windows 10 Pro 21H2 (build 19044), installed 28 Mar 2022
Computer name	DESKTOP-IFT71NV
Registered owner	Charles Kingston
Local accounts	Administrator (500, *disabled*), Guest (501, disabled), DefaultAccount (503, disabled), WDAGUtilityAccount (504), **Charles Kingston (RID 1001)**
Profile	`C:\Users\Charles Kingston`, SID S-1-5-21-875827132-2921456848-1717053428-1001
Logons	8, last logon 29 Mar 2022 07:32:53 UTC

a. Username: **`Charles Kingston`**. This is the only enabled interactive account, and it owns the profile with all the user activity.

b. Administrator privileges: yes. The `Administrators` alias (`SAM\Domains\Builtin\Aliases\00000220`, value `C`) has two members: `…-500` (the built-in Administrator, which is disabled) and **`…-1001` (Charles Kingston)**. The account's ACB flags (`0x0214`) also include *Password-not-required* (0x0004).

Why this is a risk: - The user can install any software. He installed OpenStego into his profile and could install VPNs or remote-access tools without approval. - He can disable Defender, logging and auditing, and can clear event logs. - He can make system-wide changes and create hidden accounts or persistence. - Malware or a phishing payload running in his session runs with full rights. - An auditor can no longer trust local evidence, because the user could alter it. - This breaks the principle of **least privilege** (ISO 27001 A.8.2 / CIS Control 5 & 6 / NIST SP 800-53 AC-6). An admin account with no password requirement makes it worse.

3. Internet history

Autopsy: *Data Artifacts → Web History / Web Search / Web Form Autofill / Web Accounts*. The primary browser is **Mozilla Firefox 98.0.2 (Greek locale)**, installed 29 Mar 2022 14:21 UTC from Microsoft Edge. The history file is `Users\Charles Kingston\AppData\Roaming\Mozilla\Firefox\Profiles\yrqx6htv.default-release\places.sqlite`.

CLI check:

```
        sqlite3 places.sqlite "select datetime(v.visit_date/1000000,'unixepoc
h'),p.url,p.title
          from moz_historyvisits v join moz_places p on p.id=v.place_id order
by v.visit_date;"
```

Timeline (UTC, 29 Mar 2022):

Time	Activity
14:48	Logs in to Gmail as **kingston.charles17@gmail.com**
14:58	CNN article: *“Canada F35 jets: Ottawa edges closer to deal with Lockheed Martin”* (military procurement)
14:59–15:00	Searches “knossos archaeological museum”, “knossos archaeological site”
15:01	“google maps knossos” → Google Maps, Knossos 714 09
15:01:30	**Google Maps search “Hotels”** around Knossos / Heraklion
15:01:49	**Infinity City Boutique Hotel** (35.3412, 25.1365)
15:01:54	**ibis Styles Heraklion Central** (35.3408, 25.1356)
15:02:01	**Legacy Gastro Suites** (35.3369, 25.1364)
15:02:11–15:03:12	**Galaxy Hotel Iraklio**, Leoforos Dimokratias (35.3305, 25.1383): viewed repeatedly
15:03:34–15:03:56	**Directions from *Galaxy Hotel Iraklio* → *Dedalou, Iraklio 714 09*** (Knossos area)
15:04	Searches “openstego”, visits openstego.com and GitHub, downloads `Setup-OpenStego-0.8.4.exe`
15:43–15:49	Creates a Firefox Account / Sync with kingston.charles17@gmail.com, uses the password manager
16:04–16:06	Saves logins for facebook.com (`charlie_king@gmail.com`) and mail.google.com (`kingston.charlie17@gmail.com`) (*formhistory.sqlite*, *logins.json*)
15:52	YouTube music

b. Hotel searches. The suspect looked at four hotels in Heraklion: **Infinity City Boutique Hotel, ibis Styles Heraklion Central, Legacy Gastro Suites and Galaxy Hotel Iraklio**. **Galaxy Hotel Iraklio (Leoforos Dimokratias, Heraklion)** is the strongest lead. It is the hotel he spent the most time on, and he then asked Google Maps for a route **starting at the**

Galaxy Hotel to Dedalou street near Knossos. That is what someone staying there would do. Investigators should check its guest register first. This matters because the USB stick was hidden in his hotel room.

c. Private browsing / VPN. - **What the evidence shows:** there are no VPN clients (OpenVPN, WireGuard, NordVPN, ProtonVPN), no Tor Browser and no proxy settings in the program list, Uninstall keys, Prefetch or the file system. The only network adapter is the VirtualBox NAT adapter (10.0.2.15). - **What it cannot show:** Firefox *private windows* write nothing to `places.sqlite`, so private browsing **cannot be ruled out**. Its use would show only as gaps in the history, or as traces in pagefile, hiberfil, DNS cache or memory. The Firefox account, Sync and saved passwords also mean that data may live in the cloud rather than on the disk. - **Impact on an audit of internet-usage policy:** private browsing, VPNs and Tor make **endpoint history incomplete**. Proxy, DLP and web-filter logs cannot see the destinations either, so the auditor cannot show compliance with acceptable-use policy (AUP). Controls should therefore not rely on endpoint history alone: - Disable InPrivate/private mode through GPO. - Block unapproved VPN and Tor at the firewall and with application control. - Force all traffic through a logging proxy or TLS-inspecting gateway. - Send DNS and proxy logs to a SIEM. - Record any remaining gaps as a limitation of scope in the audit report.

4. Operational security and secure communications

Check	Finding
i. Encrypted messaging (Signal, ProtonMail, Telegram, WhatsApp)	**None found.** No installs, no Prefetch, no AppData folders, no visits to these sites. Communication was through **Gmail** (kingston.charles17@gmail.com) and **Facebook**.
ii. Steganography / encryption searches	**Yes.** Firefox search **“openstego”** (15:04:13). Visits to openstego.com and the GitHub releases page. Download of `Setup-OpenStego-0.8.4.exe` (in `Downloads`, Firefox download annotation, Zone.Identifier). Prefetch files `SETUP-OPENSTEGO-0.8.4.EXE-E44648D2.pf` / `.TMP-43D9A23D.pf` prove it ran. The program was installed per-user to `AppData\Local\Programs\OpenStego`. He also created an **AES-256-encrypted ZIP** (`Pictures\vacation.zip`).
iii. Dark web / anonymity tools	**None found.** No Tor Browser, no `.onion` URLs, no VPN or proxy software.

Conclusion: the evidence supports the suspicion that he was trained in *hiding data in plain sight*. He combined **steganography (OpenStego)**, **strong archive encryption** and **anti-forensic deletion** (the password file was

“deleted” on the USB stick). He used ordinary mainstream services (Gmail, Facebook, a tourist-looking “vacation” archive) as cover, instead of conspicuous tools such as Tor or Signal.

b. Recommendations for security awareness training: 1. Teach staff that **innocuous files can carry hidden data**: steganography in images, encrypted ZIPs, “vacation” photos. Back this up with DLP rules that flag encrypted archives and images leaving the organisation. 2. **Least privilege:** explain why users must not have admin rights or install unvetted tools, and how to request software. 3. **Password hygiene:** no plaintext password files, on USB sticks or anywhere else. Use an approved password manager and MFA. 4. **Removable media:** encrypted, company-issued USB devices only. Report lost media. Understand that “Delete/Trash” does not erase data. 5. **Social engineering / insider-threat awareness:** recognise and report unusual behaviour, such as drone photos of infrastructure, or interest in military news alongside data-hiding tools. 6. **Travel security:** clean devices for travel, and what to expect at border or airport inspections. 7. Run regular phishing and steganography exercises, and measure the results. Map the training to ISO 27001 A.6.3 and NIST SP 800-50.

5. Installed software and unauthorised applications

Sources: SOFTWARE hive `Microsoft\Windows\CurrentVersion\Uninstall` (both views), `Program Files`, `AppData\Local\Programs`, Prefetch, UserAssist. In Autopsy: *Data Artifacts → Installed Programs*, *Run Programs*, *Recent Documents*.

Program	Version	Installed (UTC)	Assessment
Microsoft Edge / Edge Update	99.0.1150.55	28 Mar 2022	built-in
Microsoft Update Health Tools	3.66	28 Mar 2022	built-in
Mozilla Firefox (x64 el) + Maintenance Service	98.0.2	29 Mar 2022 14:33	user-installed browser. Acceptable only if whitelisted
OpenStego	**0.8.4**	**29 Mar 2022 ~15:05**	**High-risk / unauthorised.** Steganography tool installed **per-user in `%LOCALAPPDATA%\Programs\OpenStego`**. This bypasses `Program Files` and does not appear in the HKLM Uninstall key
Oracle VM VirtualBox Guest Additions	6.1.32	29 Mar 2022 16:12	shows the “laptop” is actually a **virtual machine**

a. Unauthorised / high-risk: OpenStego. Its only purpose is hiding and extracting data inside images, plus watermarking. Installing it per-user is a common way to get around software-installation controls.

b. Remote access / encryption tools: - **No remote-access tools** were found: no AnyDesk, TeamViewer, RustDesk, VNC or RDP wrappers, and no traces in Prefetch or the file system. - **Encryption / data hiding:** OpenStego, which supports AES-128/256 encryption of the hidden payload, and the AES-256-encrypted `vacation.zip`. - These artefacts point to covert exfiltration through images rather than through remote control.

c. Preventing unauthorised installs in a corporate environment: - Remove local admin rights. Use LAPS for the built-in administrator. - **Application allow-listing:** Microsoft AppLocker / WDAC, with rules that also cover `%LOCALAPPDATA%` and `%TEMP%` so that per-user installs are blocked. - Use a managed software catalogue (Intune, SCCM, Company Portal) with a formal change-management and approval process. - EDR/AV alerts on new executables and on known dual-use tools (stego, Tor, remote access). - Block downloads of executables at the web proxy. Turn on SmartScreen in block mode. - Run periodic software-inventory audits against the approved baseline, with a SIEM alert on Event IDs 11707/1033 (MSI installs) and 4688 (process creation). - Write all of this into an Acceptable Use and Software policy, with sanctions (ISO 27001 A.8.19 “Installation of software on operational systems”, CIS Control 2).

6. Data protection and encryption compliance

a. Encrypted files: - Autopsy *Encryption Detection* module and *Analysis Results → Encryption Detected*. - CLI: `7z l -slt vacation.zip`.

File	Detail
`C:\Users\Charles Kingston\Pictures\vacation.zip` (17,130,603 B, modified 29 Mar 2022 07:42 UTC; MD5 1c2527b43e793294a61f6430e65269b4)	**Password-protected ZIP, AES-256 + Deflate.** Holds `vacation.jpg` (13,089,579 B) and `knossos.png` (4,082,607 B). Opened recently (`Recent\vacation.lnk` → `..\Pictures\vacation.zip`).
`knossos.png` (inside the ZIP)	Candidate **steganographic carrier**. PNG is OpenStego’s output format, and the file was created the same day OpenStego was used. See Part 3.

b. BitLocker / VeraCrypt / TrueCrypt: - **None.** No VeraCrypt or TrueCrypt programs, drivers or `.hc`/`.tc` containers were found. - The partition table (`mmls laptop.E01`) shows: - a 50 MB System Reserved NTFS partition; - the main NTFS partition at sector 104448; - a 509 MB Windows Recovery partition (type 0x27). - None of them carries a BitLocker signature (`-FVE-FS-`). Both NTFS volumes open directly in Autopsy, so the volumes are **not encrypted**. - Only the standard Windows BitLocker binaries are present.

c. Policies needed when sensitive data is encrypted: - **Encryption and key-management policy:** - Use only approved algorithms and tools (BitLocker with TPM+PIN, managed 7-Zip/AES). - Escrow keys centrally (AD/Entra ID, a KMS or HSM) so the organisation can always recover its own data. - Ban private encryption keys or tools on company data. - References: ISO 27001 A.8.24, NIST SP 800-57, NIST SP 800-111. - **Data classification and handling:** label data, and require encryption at rest and in transit for Confidential or higher. GDPR Art. 32 names encryption as a “state of the art” measure, and Art. 34(3)(a) exempts breach notification when the lost data was encrypted. - **Full-disk encryption on all endpoints and removable media**, enforced through MDM/GPO. Check compliance continuously. The suspect’s laptop and USB stick were both unencrypted at disk level. - **DLP and monitoring:** detect encrypted archives and steganography leaving the network. Encrypted content that cannot be inspected should be quarantined or need approval. - **Acceptable-use and monitoring consent policy:** lets the organisation lawfully inspect corporate devices (GDPR Art. 6/88 legal basis, transparency). - **Incident response and forensic readiness:** procedures to preserve and decrypt evidence, and a chain of custody (ISO 27035, ISO 27037, NIST SP 800-86). - **Secure disposal:** NIST SP 800-88 sanitisation of media (see Part 2 Q7). - **Regular audit and review** of encryption coverage and key escrow (NIST CSF PR.DS-1/PR.DS-2, CIS Control 3).

Screenshots

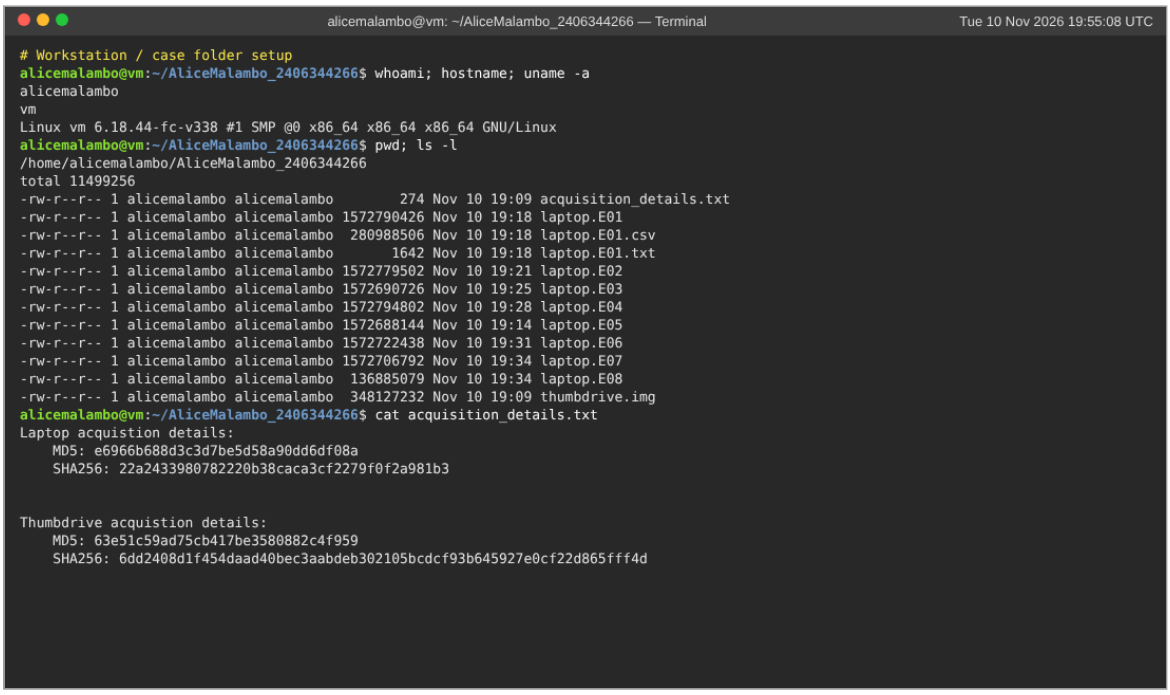


Figure 1: Workstation / case folder setup

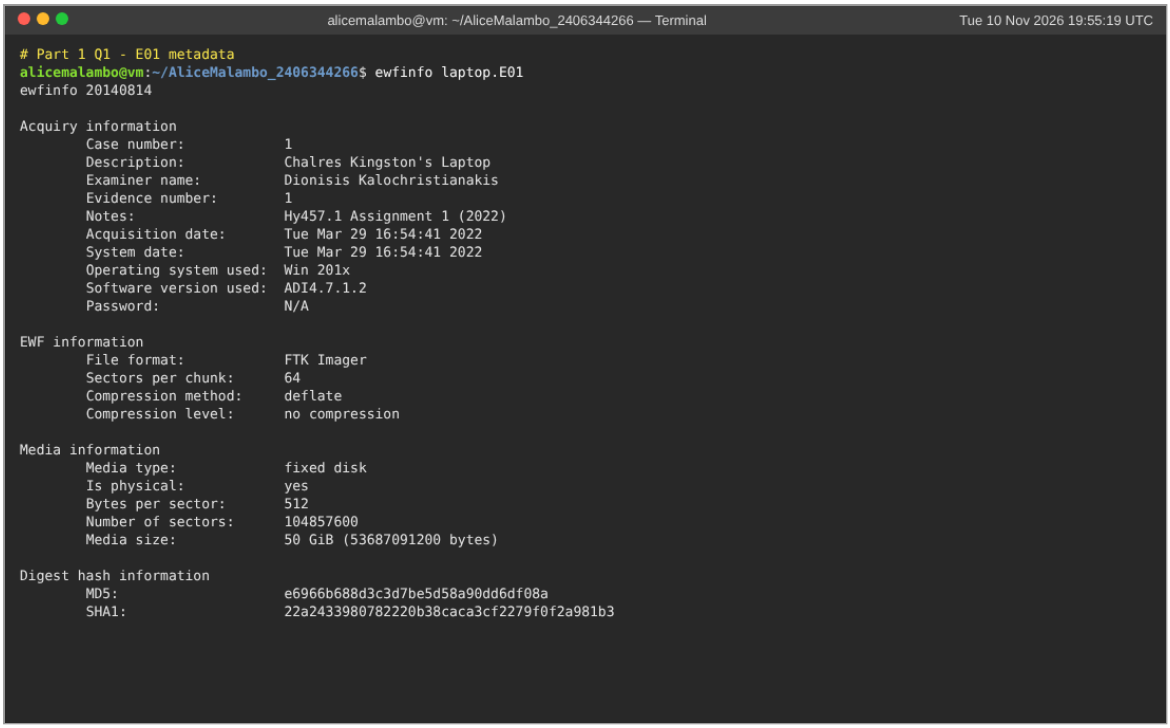


Figure 2: Part 1 Q1 - E01 metadata

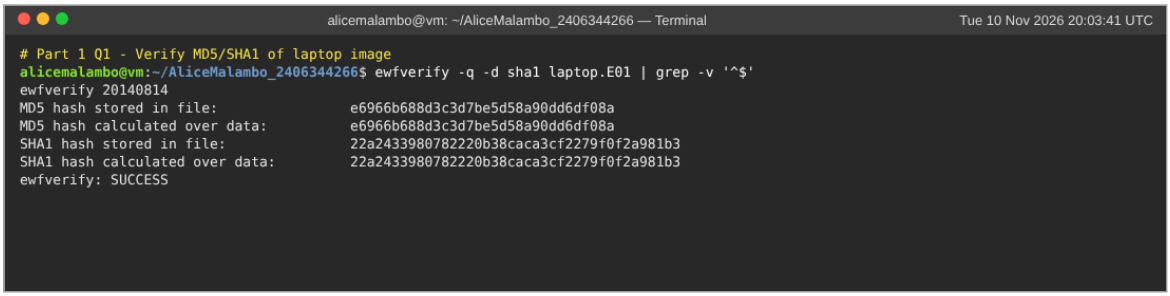


Figure 3: Part 1 Q1 - Verify MD5/SHA1 of laptop image

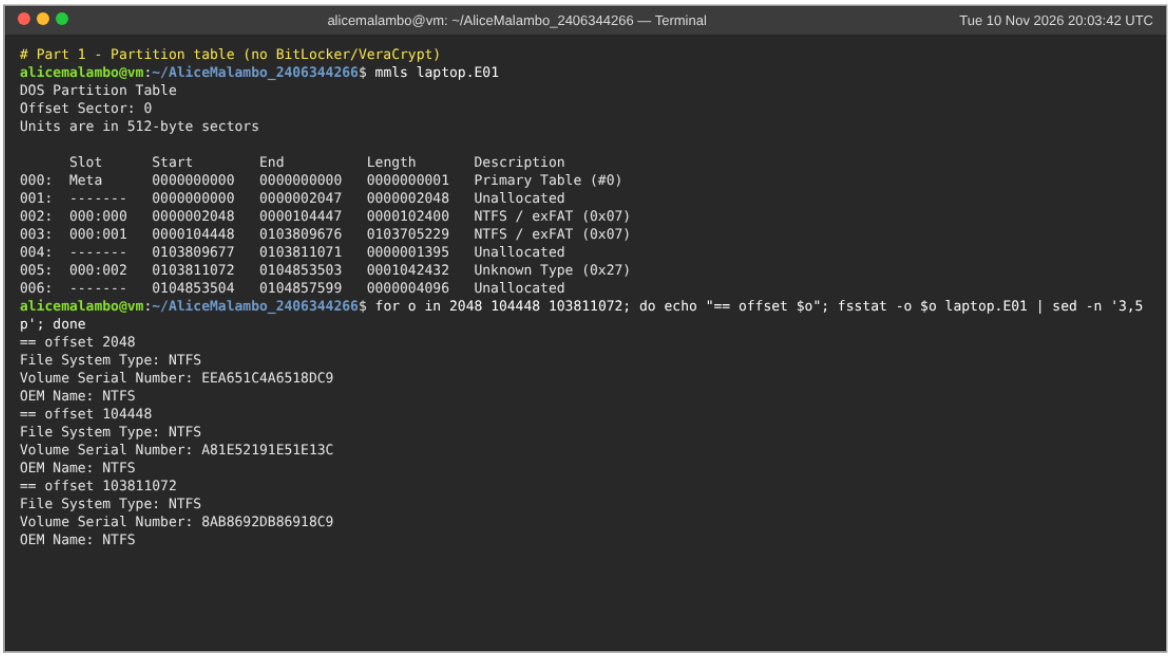


Figure 4: Part 1 - Partition table (no BitLocker/VeraCrypt)

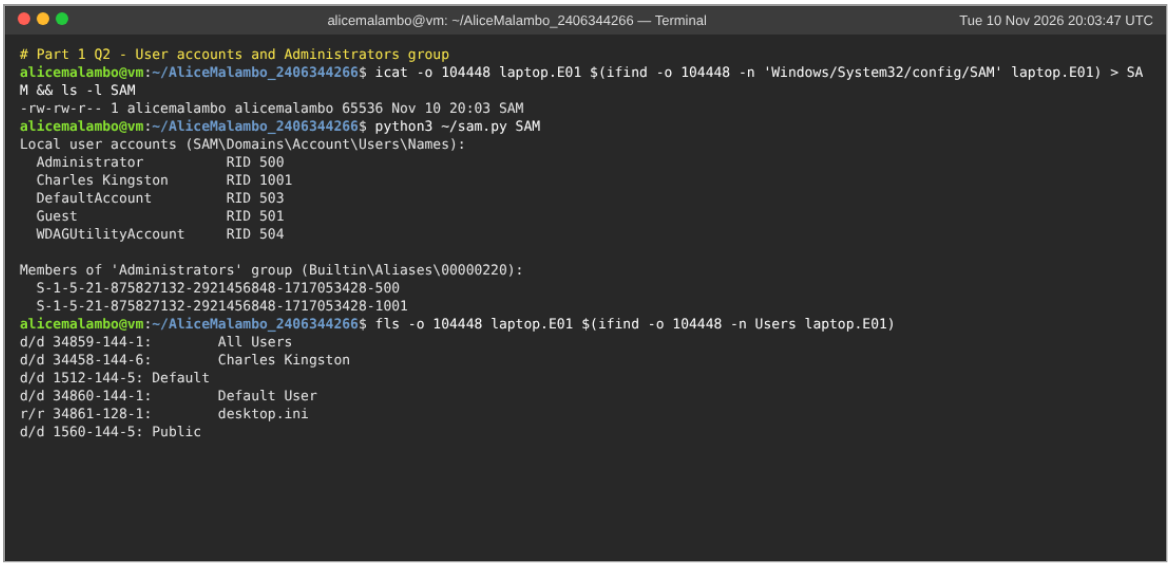


Figure 5: Part 1 Q2 - User accounts and Administrators group

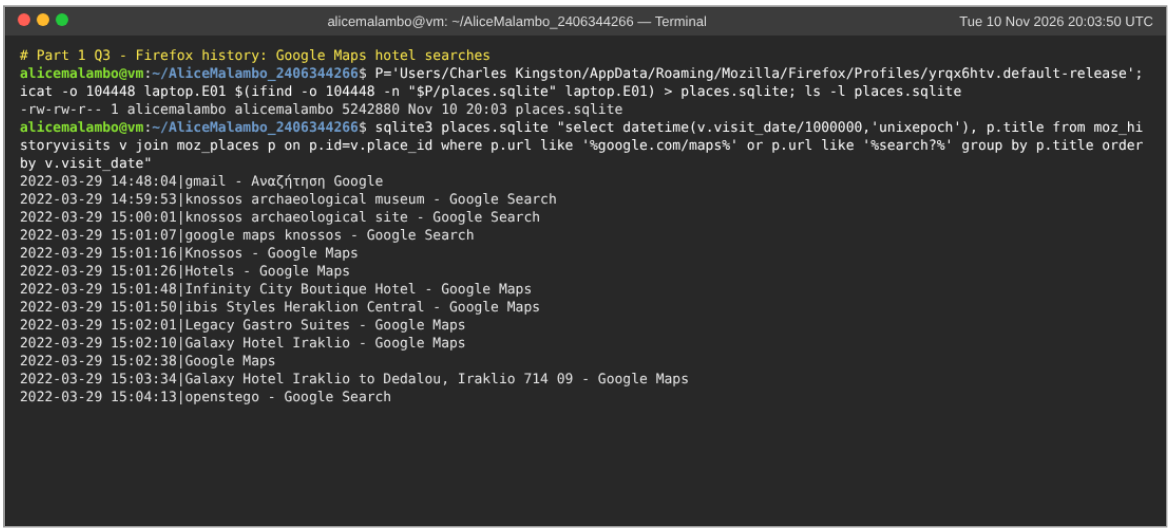


Figure 6: Part 1 Q3 - Firefox history: Google Maps hotel searches

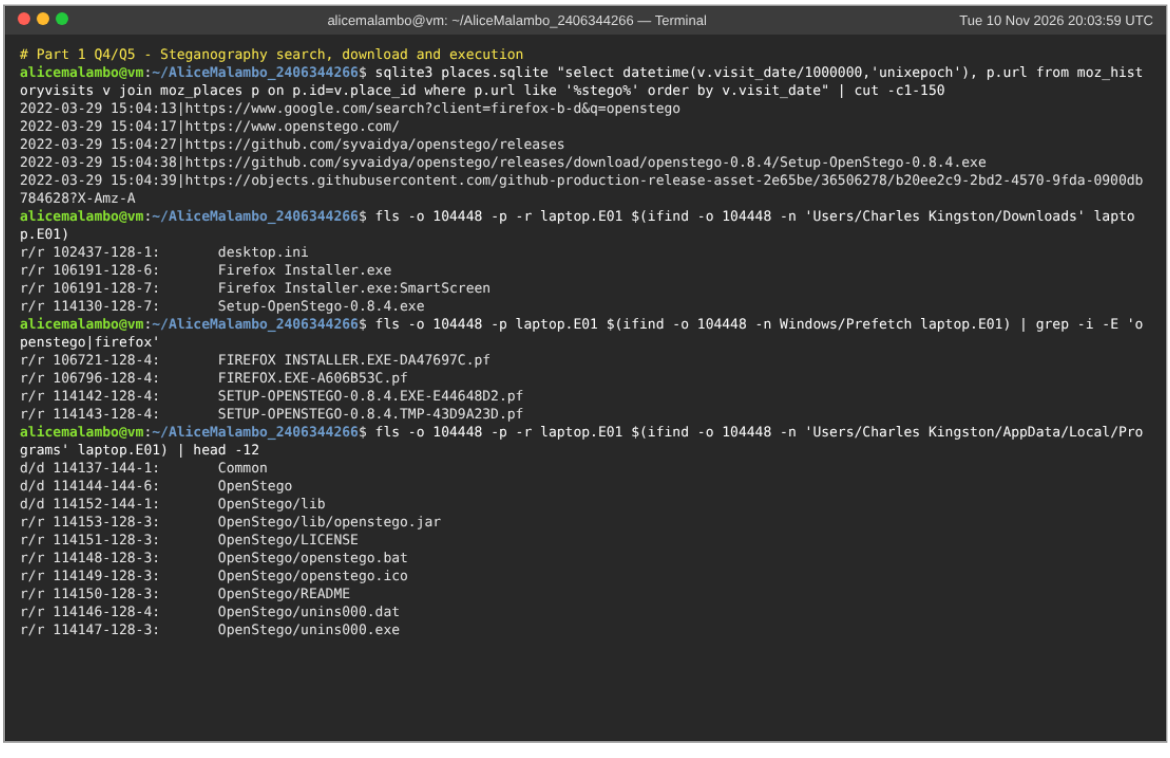


Figure 7: Part 1 Q4/Q5 - Steganography search, download and execution

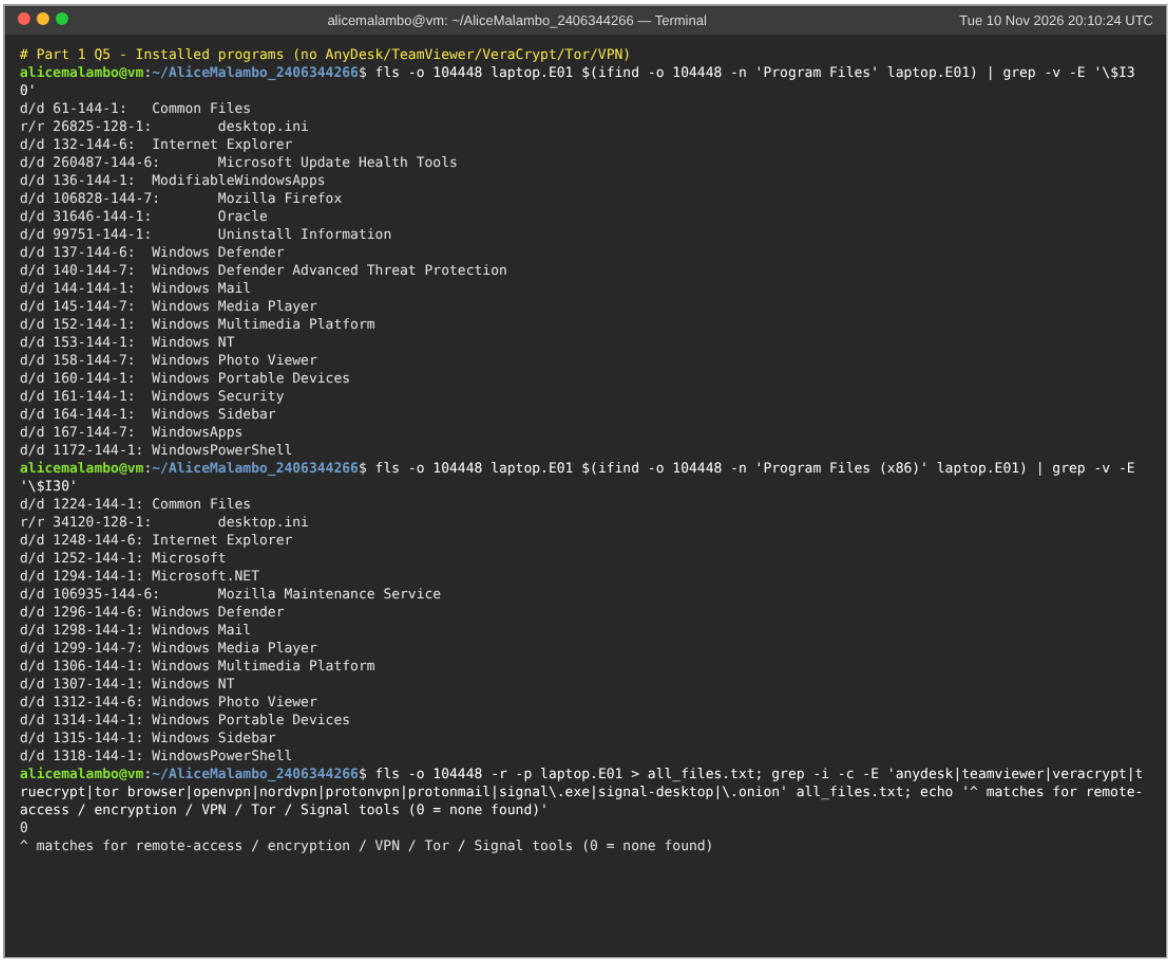


Figure 8: Part 1 Q5 - Installed programs (no AnyDesk/TeamViewer/VeraCrypt/Tor/VPN)

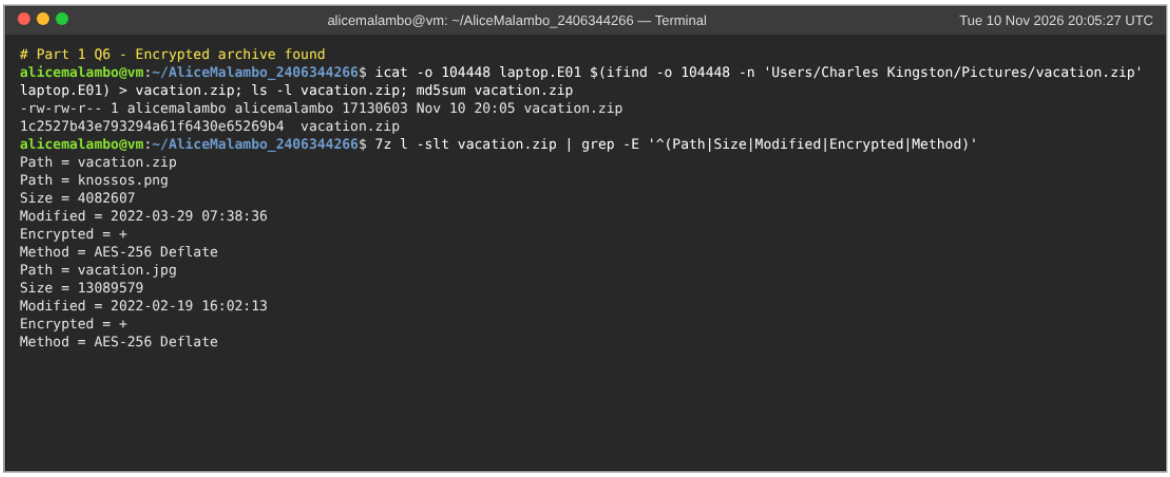


Figure 9: Part 1 Q6 - Encrypted archive found

Part 2 – FAT Analysis and File Recovery (thumbdrive.img)

> Student: Alice Malambo. Re-run every command on your own workstation (case folder `~/AliceMalambo_2406344266`) and take a screenshot of each command + output.

0. Setup and integrity check

```
mkdir ~/AliceMalambo_2406344266 && cd ~/AliceMalambo_2406344266
md5sum thumbdrive.img
sha256sum thumbdrive.img
```

	Computed	
MD5	63e51c59ad75cb417be3580882c4f959	63e51 ✅
SHA256	6dd2408d1f454daad40bec3aabdeb302105bcdcf93b645927e0cf22d865fff4d	same

1. fsstat

```
fsstat thumbdrive.img
```

Relevant output:

```
File System Type: FAT32
...
File System Layout (in sectors)
Total Range: 0 - 1001471
* Reserved: 0 - 31
** Boot Sector: 0
** FS Info Sector: 1
** Backup Boot Sector: 6
* FAT 0: 32 - 1015
* FAT 1: 1016 - 1999
* Data Area: 2000 - 1001471
** Cluster Area: 2000 - 1001471
*** Root Directory: 2000 - 2007
...
Sector Size: 512
Cluster Size: 4096
Total Cluster Range: 2 - 124935
```

Question	Answer
i. File system type	**FAT32**
ii. Sector size	**512 bytes**
iii. Cluster size	**4096 bytes** (8 sectors per cluster)
iv. FAT 0	**sectors 32 – 1015** (984 sectors)
v. Root directory	**sectors 2000 – 2007** (one cluster, cluster #2)
vi. Data area	**sectors 2000 – 1001471**
vii. Last sector of the file system	**1001471**

Note: fsstat also reports `Total Range in Image: 0 - 679935`. The file system declares 1,001,472 sectors but the image only contains 679,936 (348,127,232 bytes / 512). The image is shorter than the file system it holds. Everything in use lives at the start of the volume, so no evidence is lost.

2. Extract the first sector of the root directory

```
        dcfldd if=thumbdrive.img of=root_sector.bin bs=512 skip=2000 count=1
        xxd root_sector.bin | head -8
```

```
00000000: e570 0061 0073 0073 002e 000f 00ba 7400  .p.a.s.s......t.   <- LFN e
ntry (deleted)
00000010: 7800 7400 0000 ffff ffff 0000 ffff ffff  x.t.............
00000020: e541 5353 2020 2020 5458 5420 0071 5b99  .ASS    TXT .q[.   <- 8.3 e
ntry (deleted)
00000030: 7b54 7b54 0000 108c 7854 0300 5500 0000  {T{T....xT..U...
00000040: 412e 0054 0072 0061 0073 000f 00e4 6800  A..T.r.a.s....h.   <- LFN f
or .Trash-1000
00000050: 2d00 3100 3000 3000 3000 0000 0000 ffff  -.1.0.0.0.......
00000060: 5452 4153 482d 7e31 2020 2010 0087 6999  TRASH-~1   ...i.   <- direc
tory .Trash-1000
00000070: 7b54 7b54 0000 6999 7b54 0400 0000 0000  {T{T..i.{T......
```

i. Command: `dcfldd if=thumbdrive.img of=root_sector.bin bs=512 skip=2000 count=1`

ii. **Yes, there is a deleted file.** The 8.3 directory entry starts at **byte offset 0x20 (32 decimal)** of the root directory sector. Its first byte is `0xE5` (unallocated), followed by `ASS TXT`, so it was `PASS.TXT`. Ignoring LFN entries as instructed, it is the first 32-byte entry that is not an LFN entry (attribute byte 0x0F). Absolute byte offset in the image: 2000 × 512 + 32 = **1,024,032**.

iii. Size: bytes `55 00 00 00` at entry offset 0x1C, read little-endian, give 0x00000055 = **85 bytes**.

iv. Clusters: ⌈85 / 4096⌉ = **1 cluster**. First cluster: high word at 0x14 is `0x0000` and low word at 0x1A is `0x0003`, so **cluster 3**. Cluster 3 is sector 2000 + (3 − 2) × 8 = **sector 2008**.

(Note: Table 1 in the assignment has the offsets slightly wrong. In the real FAT32 layout, attributes are at 0x0B, the high cluster word at 0x14, the low cluster word at 0x1A and the 4-byte size at 0x1C. The values above use the real layout and were confirmed with `istat thumbdrive.img 4`.)

3. Modify the directory entry

Replace the deletion marker `0xE5` with the original first character `P` (`0x50`). You can also restore the LFN entry's sequence byte (`0xE5` → `0x41`, meaning "sequence 1, last entry") so the long name `pass.txt` comes back too.

```
        cp root_sector.bin root_sector_mod.bin
        printf '\x50' | dd of=root_sector_mod.bin bs=1 seek=32 conv=notrunc
# 8.3 entry
        printf '\x41' | dd of=root_sector_mod.bin bs=1 seek=0  conv=notrunc
# LFN entry (optional)
        xxd root_sector_mod.bin | head -4
```

```
00000000: 4170 0061 0073 0073 002e 000f 00ba 7400  Ap.a.s.s......t.
00000020: 5041 5353 2020 2020 5458 5420 0071 5b99  PASS    TXT .q[.
```

(A hex editor such as `hexedit`, `ghex` or HxD works just as well.)

4. Extract FAT 0 and fix the cluster chain

```
        dcfldd if=thumbdrive.img of=fat0.bin bs=512 skip=32 count=984
        xxd fat0.bin | head -3
```

```
00000000: f8ff ff0f ffff ff0f f8ff ff0f ffff ff0f  ................
00000010: ffff ff0f ffff ff0f ffff ff0f ffff ff0f  ................
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
```

i. **The chain is 1 cluster long.** The file is 85 bytes, so the only entry needed is FAT[3] = end-of-chain (`0x0FFFFFFF`). FAT[3] is at byte offset 3 × 4 = **12 (0x0C)** in the FAT.

ii. Change: write `FF FF FF 0F` at offset 12 of the FAT:

```
        cp fat0.bin fat0_mod.bin
        printf '\xff\xff\xff\x0f' | dd of=fat0_mod.bin bs=1 seek=12 conv=notru
nc
        cmp fat0.bin fat0_mod.bin
```

Observation: FAT[3] already contained `FF FF FF 0F`, so the FAT needed **no change**. The file had not been destroyed; it had been *moved to the Linux trash* (`.Trash-1000/files/pass.txt`, deletion date 2022-03-27 22:11:19 according to `.Trash-1000/info/pass.txt.trashinfo`). A move on FAT creates a new directory entry in `.Trash-1000/files` that points at the **same cluster 3**, and marks the old root entry as deleted. Cluster 3 is therefore still allocated. `istat thumbdrive.img 4` and `istat thumbdrive.img 518` both report sector 2008.

5. Build the new image

```
        dcfldd if=thumbdrive.img of=part1.bin bs=512 count=2000          # sec
tors 0-1999 (boot, FATs)
        dcfldd if=thumbdrive.img of=part3.bin bs=512 skip=2001           # sec
tors 2001-end
        cat part1.bin root_sector_mod.bin part3.bin | dcfldd of=thumbdrive_rec
overed.img bs=512
        cmp -l thumbdrive.img thumbdrive_recovered.img                    # sh
ows only the 2 changed bytes
```

```
1024001 345 101     (0xE5 -> 0x41, LFN entry)
1024033 345 120     (0xE5 -> 0x50, 'P')
```

(If the FAT had needed a change, you would also insert `fat0_mod.bin` at sector 32, and a copy at sector 1016 for FAT 1.)

Alternative, writing in place: `cp thumbdrive.img thumbdrive_recovered.img && dcfldd if=root_sector_mod.bin of=thumbdrive_recovered.img bs=512 seek=2000 conv=notrunc`. Warning: some dcfldd builds ignore `conv=notrunc` and truncate the output, so check the file size afterwards. This is why the concatenation method above is used.

6. Mount and verify

```
        sudo mkdir -p /mnt/AliceMalambo_2406344266_usb
        sudo mount -o loop,ro thumbdrive_recovered.img /mnt/AliceMalambo_24063
44266_usb
        ls -la /mnt/AliceMalambo_2406344266_usb
        cat /mnt/AliceMalambo_2406344266_usb/pass.txt
        sha256sum /mnt/AliceMalambo_2406344266_usb/pass.txt
```

(Verified here with `fls -p thumbdrive_recovered.img` and `mdir -i thumbdrive_recovered.img ::/`, which list `pass.txt` (85 bytes) as allocated again. On Windows you can open the image in FTK Imager or use OSFMount.)

Contents of the recovered file:

```
Facebook: 82d%x2Eg%829Wui

email: 6gT@5729

archive: QGw8rK^GM^y$%7977Sf66#85$*bcE
```

ii. **SHA256(pass.txt) = eb3142c20578ff7f00fcae00ffec9bce5fd507e653556a960a2135650019c018**

iii. **Significance:** the file holds the suspect's credentials for Facebook, for an e-mail account, and above all for an **archive** (`QGw8rK^GM^y$%7977Sf66#85$*bcE`). This is the password needed to decrypt the encrypted archive on the laptop in Part 3. The suspect "deleted" the file and hid the USB drive in the hotel room, which shows an attempt to conceal it. However, he only moved it to the trash, so the data was never overwritten.

7. Anti-forensics: secure erasure

i. **Method:**

- **Magnetic HDD:** overwrite the whole device, not individual files. Use a single pass of zeros or random data over every addressable sector, for example `dd if=/dev/urandom of=/dev/sdX bs=1M status=progress`, `shred -n 1 -z /dev/sdX`, or `nwipe`. Also wipe the HPA/DCO areas (`hdparm`).
- **SSD / USB flash:** use the device's built-in sanitize command: ATA Secure Erase / Sanitize Block Erase (`hdparm --security-erase`), NVMe Format/Sanitize (`nvme sanitize`, `nvme format -s1`), or a **cryptographic erase**

(destroy the key of a drive that was fully encrypted from day one with BitLocker/LUKS/VeraCrypt).
- For media that held classified data: physical destruction (shred, disintegrate, incinerate) or, for HDDs only, degaussing.
- Verify afterwards. Re-read the device and confirm it holds only the wipe pattern (`cmp /dev/sdX /dev/zero`, or a hex dump of random samples).

ii. **Why it works:**

- The recovery in this assignment worked because deleting a file on FAT only changes **metadata**. The first byte of the directory entry becomes `0xE5` and the FAT entries are zeroed. The **data clusters are never touched**. Overwriting every sector removes the data itself, so there is nothing left for `fls`, `icat`, or file carving tools (photorec, foremost) to find.
- On modern high-density HDDs one overwrite pass is enough. NIST SP 800-88 Rev. 1 says a single overwrite pass "is sufficient" for the *Clear* level on ATA hard drives. Wright, Kleiman & Sundhar (2008) showed that recovering overwritten bits with magnetic force microscopy is essentially no better than guessing (~50 %). This disproves the old Gutmann 35-pass myth for modern drives.
- Flash memory is different. Wear levelling, over-provisioning and bad-block remapping mean that an OS-level overwrite does not reach every physical cell. Wei et al. (USENIX FAST 2011) found that single-file overwrite on SSDs left 4–75 % of the data recoverable. Whole-device software overwrites were usually, but not always, effective. Firmware sanitize commands erase all NAND blocks, including hidden ones. Crypto-erase makes any remaining ciphertext useless because the key no longer exists (NIST SP 800-88 *Purge*; IEEE 2883-2022).
- Physical destruction is the only method that covers firmware bugs. That is why NIST requires *Destroy* for the highest-sensitivity media.

References: 1. Kissel R. et al., *NIST SP 800-88 Rev. 1 – Guidelines for Media Sanitization*, NIST, 2014. 2. Wright C., Kleiman D., Sundhar S., "Overwriting Hard Drive Data: The Great Wiping Controversy", ICISS 2008, LNCS 5352. 3. Wei M., Grupp L., Spada F., Swanson S., "Reliably Erasing Data From Flash-Based Solid State Drives", USENIX FAST 2011. 4. IEEE Std 2883-2022, *Standard for Sanitizing Storage*. 5. Carrier B., *File System Forensic Analysis*, Addison-Wesley, 2005 (Ch. 9–10, FAT). 6. Gutmann P., "Secure Deletion of Data from Magnetic and Solid-State Memory", USENIX Security 1996 (and its epilogue noting the 35 passes are unnecessary for modern drives).

Screenshots

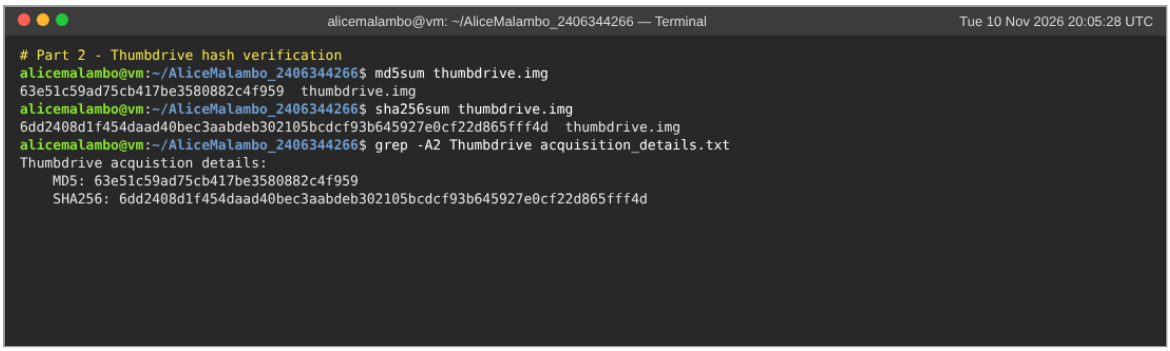


Figure 10: Part 2 - Thumbdrive hash verification

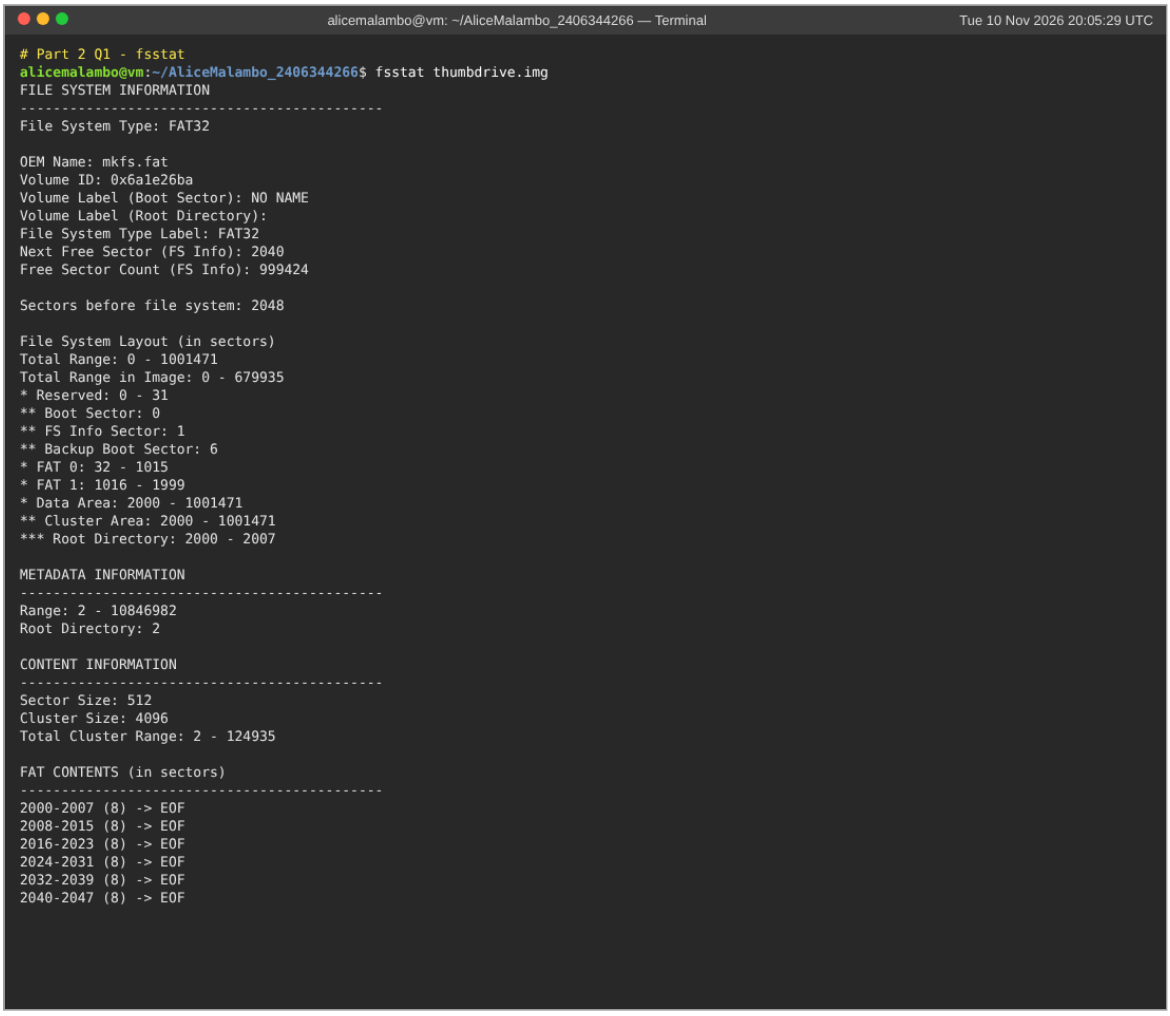


Figure 11: Part 2 Q1 - fsstat

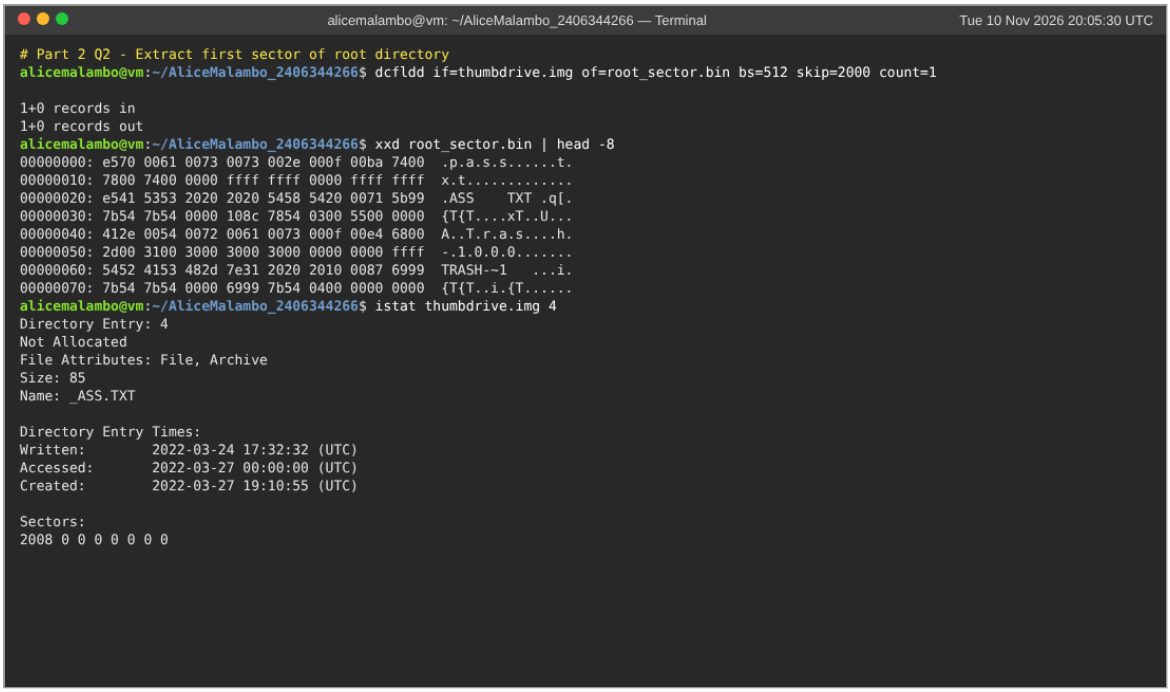


Figure 12: Part 2 Q2 - Extract first sector of root directory

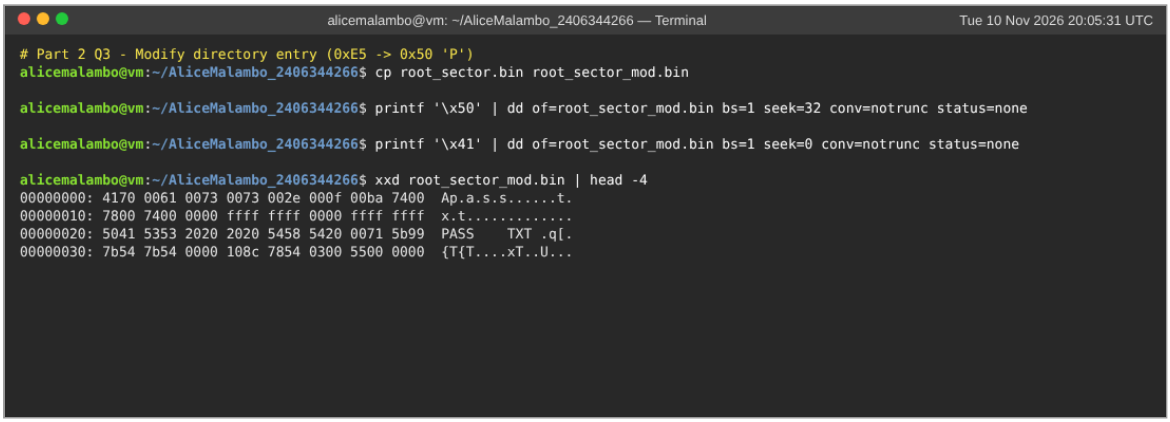


Figure 13: Part 2 Q3 - Modify directory entry (0xE5 -> 0x50 ‘P’)

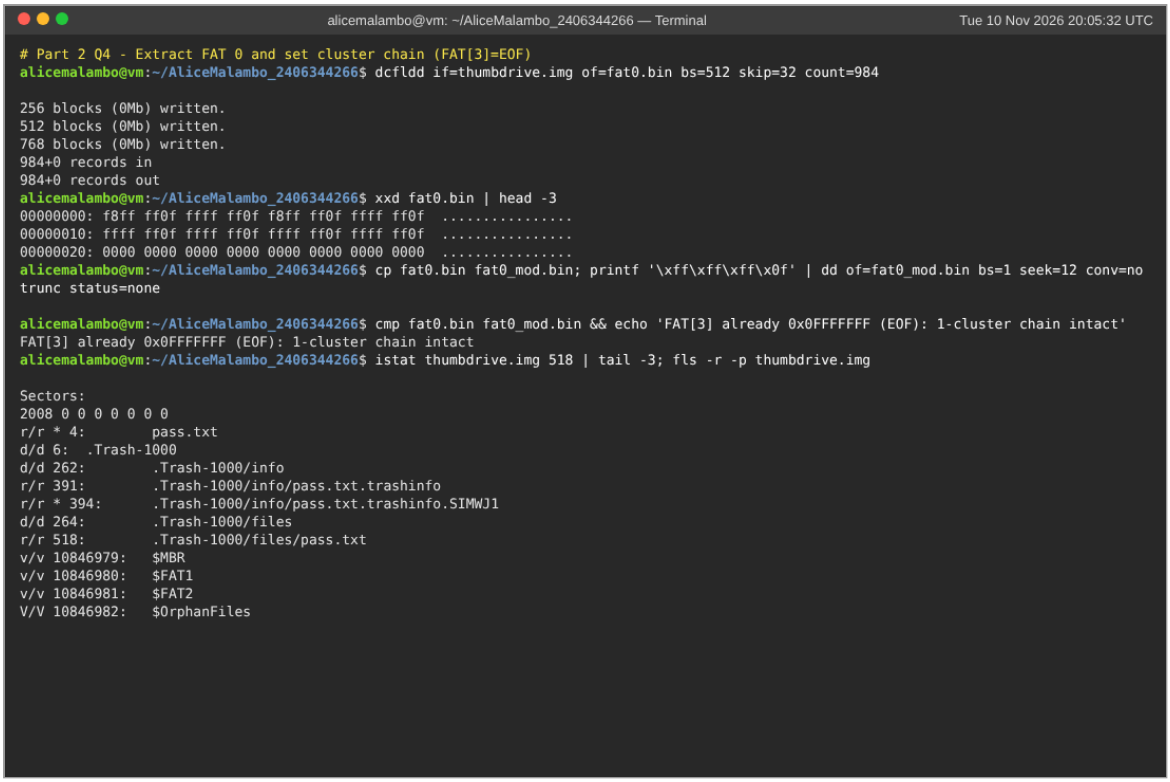


Figure 14: Part 2 Q4 - Extract FAT 0 and set cluster chain (FAT[3]=EOF)

```
# Part 2 Q5 - Recreate image with modified root directory
alicemalambo@vm:~/AliceMalambo_2406344266$ dcfldd if=thumbdrive.img of=part1.bin bs=512 count=2000

256 blocks (0Mb) written.
512 blocks (0Mb) written.
768 blocks (0Mb) written.
1024 blocks (0Mb) written.
1280 blocks (0Mb) written.
1536 blocks (0Mb) written.
1792 blocks (0Mb) written.
2000+0 records in
2000+0 records out
alicemalambo@vm:~/AliceMalambo_2406344266$ dcfldd if=thumbdrive.img of=part3.bin bs=512 skip=2001

256 blocks (0Mb) written.
512 blocks (0Mb) written.
768 blocks (0Mb) written.
1024 blocks (0Mb) written.
1280 blocks (0Mb) written.
1536 blocks (0Mb) written.
1792 blocks (0Mb) written.
2048 blocks (1Mb) written.
2304 blocks (1Mb) written.
2560 blocks (1Mb) written.
2816 blocks (1Mb) written.
3072 blocks (1Mb) written.
3328 blocks (1Mb) written.
3584 blocks (1Mb) written.
3840 blocks (1Mb) written.
4096 blocks (2Mb) written.
4352 blocks (2Mb) written.
4608 blocks (2Mb) written.
4864 blocks (2Mb) written.
5120 blocks (2Mb) written.
5376 blocks (2Mb) written.
5632 blocks (2Mb) written.
5888 blocks (2Mb) written.
6144 blocks (3Mb) written.
6400 blocks (3Mb) written.
6656 blocks (3Mb) written.
6912 blocks (3Mb) written.
7168 blocks (3Mb) written.
7424 blocks (3Mb) written.
7680 blocks (3Mb) written.
7936 blocks (3Mb) written.
8192 blocks (4Mb) written.
8448 blocks (4Mb) written.
8704 blocks (4Mb) written.
8960 blocks (4Mb) written.
9216 blocks (4Mb) written.
9472 blocks (4Mb) written.
9728 blocks (4Mb) written.
9984 blocks (4Mb) written.
10240 blocks (5Mb) written.
10496 blocks (5Mb) written.
10752 blocks (5Mb) written.
11008 blocks (5Mb) written.
11264 blocks (5Mb) written.
11520 blocks (5Mb) written.
11776 blocks (5Mb) written.
12032 blocks (5Mb) written.
12288 blocks (6Mb) written.
12544 blocks (6Mb) written.
12800 blocks (6Mb) written.
13056 blocks (6Mb) written.
13312 blocks (6Mb) written.
13568 blocks (6Mb) written.
13824 blocks (6Mb) written.
14080 blocks (6Mb) written.
14336 blocks (7Mb) written.
14592 blocks (7Mb) written.
14848 blocks (7Mb) written.
15104 blocks (7Mb) written.
... (2591 more lines)
alicemalambo@vm:~/AliceMalambo_2406344266$ cat part1.bin root_sector_mod.bin part3.bin | dcfldd of=thumbdrive_recovered.img bs=512

256 blocks (0Mb) written.
512 blocks (0Mb) written.
768 blocks (0Mb) written.
1024 blocks (0Mb) written.
1280 blocks (0Mb) written.
1536 blocks (0Mb) written.
1792 blocks (0Mb) written.
2048 blocks (1Mb) written.
2304 blocks (1Mb) written.
2560 blocks (1Mb) written.
2816 blocks (1Mb) written.
3072 blocks (1Mb) written.
3328 blocks (1Mb) written.
3584 blocks (1Mb) written.
3840 blocks (1Mb) written.
4096 blocks (2Mb) written.
4352 blocks (2Mb) written.
4608 blocks (2Mb) written.
4864 blocks (2Mb) written.
5120 blocks (2Mb) written.
5376 blocks (2Mb) written.
5632 blocks (2Mb) written.
5888 blocks (2Mb) written.
6144 blocks (3Mb) written.
6400 blocks (3Mb) written.
6656 blocks (3Mb) written.
6912 blocks (3Mb) written.
7168 blocks (3Mb) written.
7424 blocks (3Mb) written.
7680 blocks (3Mb) written.
7936 blocks (3Mb) written.
8192 blocks (4Mb) written.
8448 blocks (4Mb) written.
8704 blocks (4Mb) written.
```

```
8960 blocks (4Mb) written.
9216 blocks (4Mb) written.
9472 blocks (4Mb) written.
9728 blocks (4Mb) written.
9984 blocks (4Mb) written.
10240 blocks (5Mb) written.
10496 blocks (5Mb) written.
10752 blocks (5Mb) written.
11008 blocks (5Mb) written.
11264 blocks (5Mb) written.
11520 blocks (5Mb) written.
11776 blocks (5Mb) written.
12032 blocks (5Mb) written.
12288 blocks (6Mb) written.
12544 blocks (6Mb) written.
12800 blocks (6Mb) written.
13056 blocks (6Mb) written.
13312 blocks (6Mb) written.
13568 blocks (6Mb) written.
13824 blocks (6Mb) written.
14080 blocks (6Mb) written.
14336 blocks (7Mb) written.
14592 blocks (7Mb) written.
14848 blocks (7Mb) written.
15104 blocks (7Mb) written.
... (2599 more lines)
alicemalambo@vm:~/AliceMalambo_2406344266$ ls -l thumbdrive.img thumbdrive_recovered.img
-rw-r--r-- 1 alicemalambo alicemalambo 348127232 Nov 10 19:09 thumbdrive.img
-rw-rw-r-- 1 alicemalambo alicemalambo 348127232 Nov 10 20:05 thumbdrive_recovered.img
alicemalambo@vm:~/AliceMalambo_2406344266$ cmp -l thumbdrive.img thumbdrive_recovered.img
  1024001 345 101
  1024033 345 120
```

Figure 15: Part 2 Q5 - Recreate image with modified root directory

```
alicemalambo@vm: ~/AliceMalambo_2406344266 — Terminal                Tue 10 Nov 2026 20:05:35 UTC
# Part 2 Q6 - Verify recovery and hash recovered file
alicemalambo@vm:~/AliceMalambo_2406344266$ fls -p thumbdrive_recovered.img
r/r 4:  pass.txt
d/d 6:  .Trash-1000
v/v 10846979:   $MBR
v/v 10846980:   $FAT1
v/v 10846981:   $FAT2
V/V 10846982:   $OrphanFiles
alicemalambo@vm:~/AliceMalambo_2406344266$ mdir -i thumbdrive_recovered.img ::/
 Volume in drive : has no label
 Volume Serial Number is 6A1E-26BA
Directory for ::/

PASS     TXT        85 2022-03-24  17:32  pass.txt
TRASH-~1     <DIR>     2022-03-27  19:11  .Trash-1000
        2 files                 85 bytes
                        511 705 088 bytes free
alicemalambo@vm:~/AliceMalambo_2406344266$ mcopy -o -i thumbdrive_recovered.img ::/pass.txt pass.txt && cat pass.txt
Facebook: 82d%x2Eg%829Wui

email: 6gT@5729

archive: QGw8rK^GM^y$%7977Sf66#85$*bcE
alicemalambo@vm:~/AliceMalambo_2406344266$ sha256sum pass.txt
eb3142c20578ff7f00fcae00ffec9bce5fd507e653556a960a2135650019c018  pass.txt
```

Figure 16: Part 2 Q6 - Verify recovery and hash recovered file

Part 3 – Image Analysis (laptop.E01)

1. Decrypt the archive

Export `C:\Users\Charles Kingston\Pictures\vacation.zip` from Autopsy (right-click → *Extract File(s)*) into `AliceMalambo_2406344266/export`, then:

```
        7z l -slt vacation.zip            # shows: Method = AES-256 Deflate, E
ncrypted = +
```

```
7z x -p'QGw8rK^GM^y$%7977Sf66#85$*bcE' vacation.zip -ovacation
sha256sum vacation/*
```

(On Windows: 7-Zip → *Extract* → enter the password. The built-in Windows ZIP tool cannot open AES-encrypted archives.)

The password is the **“archive”** entry from the `pass.txt` file recovered in step 2.6: `QGw8rK^GM^y$%7977Sf66#85$*bcE`. The archive decrypts successfully:

File	Size	Modified (ZIP)	SHA-256
vacation.jpg	13,089,579 B	2022-02-19 16:02:13	fe00132122fd6823b6626c51b07756d099d01698e
knossos.png	4,082,607 B	2022-03-29 07:38:36	2debd21296f8d53cc9c9b6e14980db55f45b08b69f

2. Date of vacation.jpg

```
exiftool vacation.jpg
```

(Or Autopsy → *Data Artifacts* → *EXIF Metadata* when you add the extracted files as a *Logical Files* data source. On Windows you can also use *Properties* → *Details*.)

```
Make                : DJI
Camera Model Name   : FC3411
Date/Time Original  : 2022:02:19 17:16:53
Create Date         : 2022:02:19 17:16:53
GPS Latitude        : 35.330057 N
GPS Longitude       : 24.998574 E
GPS Altitude        : 498.9 m Above Sea Level
Image Size          : 5472x3648
```

The photo was taken on 19 February 2022 at 17:16:53 (camera local time). This is more than a month **before** the suspect’s laptop activity (28–29 March 2022), so he had been in Crete, or had received the photo, well before his “tourist” trip was supposedly taking place.

The camera is a **DJI FC3411**, the camera module of the **DJI Air 2S drone**. The picture is an **aerial drone photograph**, not a holiday snapshot.

3. Location of vacation.jpg

GPS: 35.330057 N, 24.998574 E, altitude ≈ 499 m.

Google Maps link to open and screenshot: **https://www.google.com/maps?q=35.330057,24.998574** (type `35.330057, 24.998574` in the Google Maps search box, switch to *Satellite* view, and take the screenshot with your

browser window visible).

Interpretation: - The point is in the **hills about 13 km west of Heraklion, Crete**, in the mountainous area west of the city. - The photo shows, from about 500 m altitude: - a **newly paved road**, with fresh cut-and-fill and a drainage culvert; - two dirt tracks; - a small isolated building; - barren limestone hills. - It contains nothing a tourist would photograph, such as an archaeological site, a beach or a town. - It is a deliberate **aerial survey of a remote road and infrastructure**, the kind of reconnaissance image an intelligence officer would collect. - Crete hosts important NATO and Greek military installations, so infrastructure imagery from the island is of interest to foreign services.

Map confirmation (Figures 20–22). The GPS coordinates were plotted on satellite imagery (Esri World Imagery): - **Figure 20** places the point in the hills of the Malevizi area, **about 6 km west of Gazi and 13 km west of Heraklion (Iraklio) city centre**. - **Figure 21** shows the same scene as the drone photo: - the curved **paved road** running east–west; - the **parallel oval dirt tracks** just south of it; - the **small white building** at the western end of the tracks; - the barren limestone hillside to the north. - This confirms that `vacation.jpg` was taken by a drone hovering over this spot, looking at the road. - Alternative: Google Maps, https://www.google.com/maps?q=35.330057,24.998574 (Satellite view).

4. Other pictures

Image	Findings
knossos.png (1364×1600, 4.1 MB)	Looks like an ordinary tourist photo of the **North Entrance / Charging Bull fresco at the Palace of Knossos**. Several things make it suspicious: • It is a **PNG**, the only lossless output format of OpenStego, and unusual for a camera photo. • At 4.1 MB it is very large for its resolution. • It has **no EXIF/camera metadata**, so it was re-saved by software. • Its timestamp is 29 Mar 2022, the same day he searched for, downloaded and installed **OpenStego 0.8.4**. • It was hidden inside the **AES-encrypted** archive together with the drone photo. This is very likely the **steganographic carrier** holding a hidden message.
Other pictures on the laptop	Only Windows Store tile logos and three small Firefox new-tab thumbnails. Nothing of interest.

How to check knossos.png on your workstation (OpenStego is free):

1. Install OpenStego 0.8.4 (needs Java) from https://www.openstego.com.
2. *Extract Data* → Input Stego File = `knossos.png`, Output Folder = `AliceMalambo_2406344266/stego`.
3. Leave the password **empty** first, then try the passwords from `pass.txt`.
4. Take a screenshot of the result and report the extracted file and its contents.

I tried the three passwords from `pass.txt` (archive, Facebook and email) from the command line, and none of them opened a payload. I could not test an empty password. **Run this step on your own PC.**

For a quick visual check, view the least-significant-bit plane of `knossos.png` in StegSolve or https://aperisolve.com (for a local copy, use `zsteg`). Random noise in the LSBs of the top rows is the usual signature of OpenStego's RandomLSB algorithm.

Statistical LSB check (own Python/NumPy script, no third-party stego tool): - In the smooth sky at the top of the image, 71 % of adjacent pixels share the same LSB value. That is the natural pattern, and suggests no sequential (top-down) LSB embedding. - Elsewhere the LSBs are about 50 % random, which is normal for a photo derived from a JPEG. - The chi-square pairs-of-values test does not show the "evened-out" histogram typical of heavy LSB embedding. - OpenStego's RandomLSB algorithm scatters a small payload across pseudo-random pixels chosen by the password, so a short hidden message would not be detectable this way. - The statistical check is therefore **inconclusive**. Extracting with OpenStego (empty password, then the passwords from `pass.txt`) is the definitive test.

Conclusion of Part 3: - The "vacation" archive on the laptop is protected with a strong password, and that password was hidden on a USB stick in his hotel room. The archive contains: - a **covert drone reconnaissance photo** of remote infrastructure west of Heraklion, with precise GPS; - a tourist-looking **Knossos image that was very probably used as a steganographic carrier** right after he installed OpenStego. - Together with the other evidence, this is strongly inconsistent with his claim of being a tourist and supports the suspicion of espionage: - the hotel searches; - his interest in military news (the F-35 article); - the use of steganography and encryption; - the deliberate deletion of the password file.

Image previews



vacation.jpg (DJI FC3411, 19 Feb 2022, 35.330057 N 24.998574 E)



knossos.png (probable OpenStego carrier)

Screenshots

```
alicemalambo@vm: ~/AliceMalambo_2406344266 — Terminal                Tue 10 Nov 2026 20:05:36 UTC
# Part 3 Q1 - Decrypt vacation.zip with password from pass.txt
alicemalambo@vm:~/AliceMalambo_2406344266$ 7z x -y -p'QGw8rK^GM^y$%7977Sf66#85$*bcE' -ovacation vacation.zip | tail -6

Everything is Ok

Files: 2
Size:       17172186
Compressed: 17130603
alicemalambo@vm:~/AliceMalambo_2406344266$ ls -l vacation; sha256sum vacation/*
total 16772
-rw-rw-r-- 1 alicemalambo alicemalambo  4082607 Mar 29  2022 knossos.png
-rw-r--r-- 1 alicemalambo alicemalambo 13089579 Feb 19  2022 vacation.jpg
2debd21296f8d53cc9c9b6e14980db55f45b08b69f6acbe180486c3d22eed451  vacation/knossos.png
fe00132122fd6823b6626c51b07756d099d01698e6cae5c8d6819af1fdb4c367  vacation/vacation.jpg
```

Figure 17: Part 3 Q1 - Decrypt vacation.zip with password from pass.txt

```
alicemalambo@vm: ~/AliceMalambo_2406344266 — Terminal                Tue 10 Nov 2026 20:05:37 UTC

# Part 3 Q2/Q3 - Date and GPS location of vacation.jpg
alicemalambo@vm:~/AliceMalambo_2406344266$ exiftool -Make -Model -DateTimeOriginal -CreateDate -GPSLatitude -GPSLongitude -GPSAltitude -
GPSPosition -ImageSize -c '%.6f' vacation/vacation.jpg
Make                            : DJI
Camera Model Name               : FC3411
Date/Time Original              : 2022:02:19 17:16:53
Create Date                     : 2022:02:19 17:16:53
GPS Latitude                    : 35.330057 N
GPS Longitude                   : 24.998574 E
GPS Altitude                    : 498.9 m Above Sea Level
GPS Position                    : 35.330057 N, 24.998574 E
Image Size                      : 5472x3648
alicemalambo@vm:~/AliceMalambo_2406344266$ echo 'Google Maps: https://www.google.com/maps?q=35.330057,24.998574'
Google Maps: https://www.google.com/maps?q=35.330057,24.998574
```

Figure 18: Part 3 Q2/Q3 - Date and GPS location of vacation.jpg

```
alicemalambo@vm: ~/AliceMalambo_2406344266 — Terminal                Tue 10 Nov 2026 20:05:38 UTC

# Part 3 Q4 - Other pictures: knossos.png metadata
alicemalambo@vm:~/AliceMalambo_2406344266$ exiftool vacation/knossos.png | grep -v -E 'Permissions|Directory|Inode'
ExifTool Version Number         : 12.76
File Name                       : knossos.png
File Size                       : 4.1 MB
File Modification Date/Time     : 2022:03:29 07:38:36+00:00
File Access Date/Time           : 2026:11:10 20:05:36+00:00
File Type                       : PNG
File Type Extension             : png
MIME Type                       : image/png
Image Width                     : 1364
Image Height                    : 1600
Bit Depth                       : 8
Color Type                      : RGB
Compression                     : Deflate/Inflate
Filter                          : Adaptive
Interlace                       : Noninterlaced
Image Size                      : 1364x1600
Megapixels                      : 2.2
```

Figure 19: Part 3 Q4 - Other pictures: knossos.png metadata

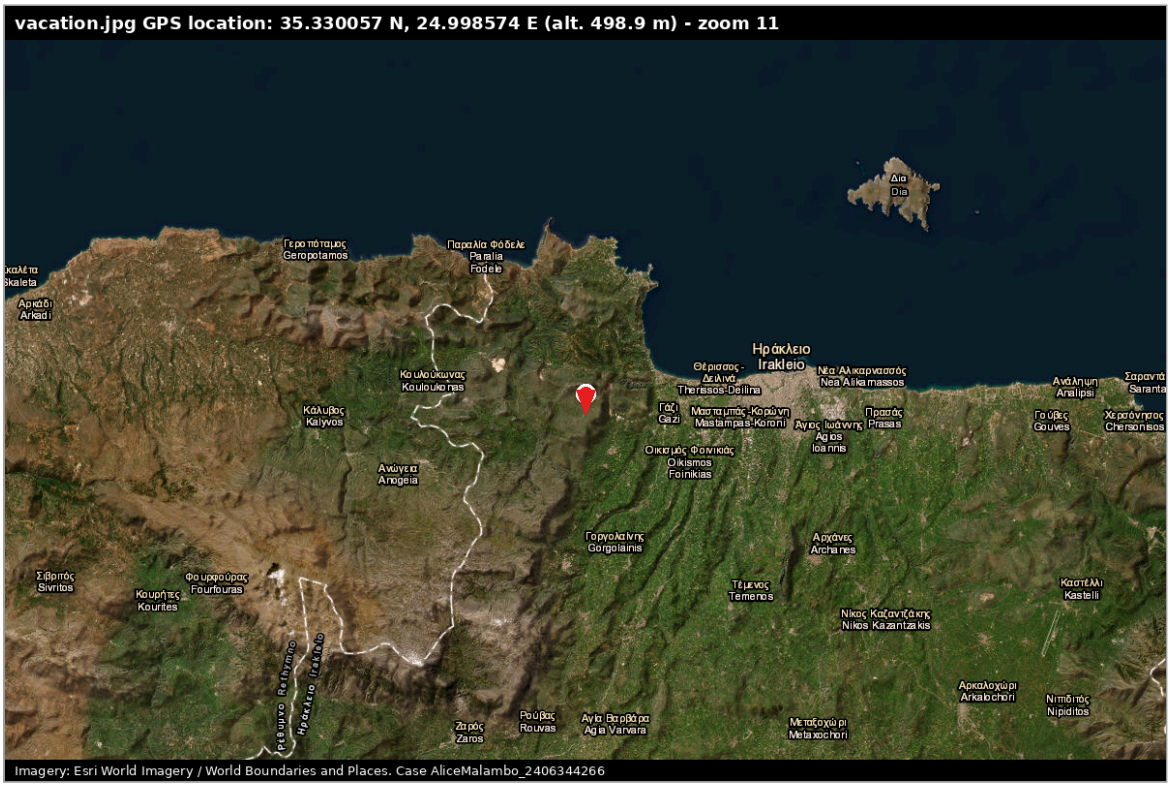


Figure 20: Map – region (vacation.jpg GPS point west of Heraklion)



Figure 21: Map – satellite view matching the drone photo

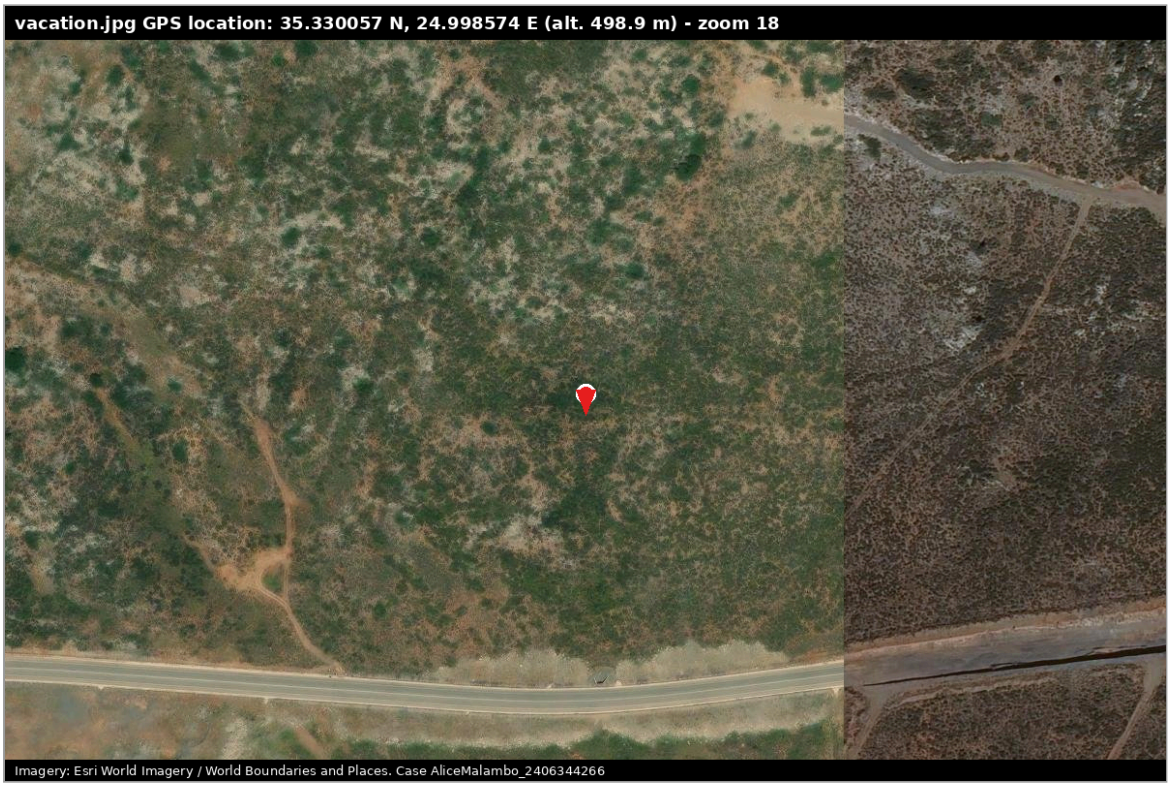


Figure 22: Map – close-up of the GPS point and road